

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:

- Explain Big O notation and how it helps in analyzing algorithms.
- Describe the best, average, and worst-case scenarios for search operations.

2. Setup:

- Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.

3. Implementation:

- Implement linear search and binary search algorithms.
- Store products in an array for linear search and a sorted array for binary search.

4. Analysis:

- Compare the time complexity of linear and binary search algorithms.
- Discuss which algorithm is more suitable for your platform and why.

Solution:

Big O Notation : it is the way to predict time taken by computer program to run how much memory it will use as its data set gets bigger

Best case, Average case, worst case in binary and linear search

	Best case	Avg case	Worst case
<u>Linear Search</u>	$O(1)$	$O(n)$	$O(n)$
<u>Binary Search</u>	$O(1)$	$O(\log n)$	$O(\log n)$

```

import java.util.*;
class Goods {
    int goodsId;
    String goodsName;
    String goodsCategory;
    Goods(int goodsId, String goodsName, String goodsCategory) {
        this.goodsId = goodsId;
        this.goodsName = goodsName;
        this.goodsCategory = goodsCategory;
    }
    void displayGoods() {
        System.out.println("Goods ID : " + goodsId);
        System.out.println("Goods Name : " + goodsName);
        System.out.println("Goods Category : " + goodsCategory);
    }
}
public class GoodsSearchSystem {
    static Goods linearFind(Goods[] goodsArray, int targetGoodsId) {
        for (Goods singleGoods : goodsArray) {
            if (singleGoods.goodsId == targetGoodsId) {
                return singleGoods;
            }
        }
        return null;
    }
    static Goods binaryFind(Goods[] goodsArray, int targetGoodsId) {
        int leftPointer = 0;
        int rightPointer = goodsArray.length - 1;

        while (leftPointer <= rightPointer) {
            int centerIndex = leftPointer + (rightPointer - leftPointer) / 2;

            if (goodsArray[centerIndex].goodsId == targetGoodsId) {
                return goodsArray[centerIndex];
            }
        }
    }
}

```

```

        if (goodsArray[centerIndex].goodsId < targetGoodsId) {
            leftPointer = centerIndex + 1;
        } else {
            rightPointer = centerIndex - 1;
        }
    }
    return null;
}
public static void main(String[] args) {
    Scanner scannerObj = new Scanner(System.in);
    System.out.println("E-Commerce Product Search");
    System.out.print("Enter number of goods: ");
    int goodsCount = scannerObj.nextInt();
    scannerObj.nextLine();
    Goods[] goodsList = new Goods[goodsCount];
    for (int index = 0; index < goodsCount; index++) {
        System.out.println("\nGoods " + (index + 1));
        System.out.print("Enter Goods ID: ");
        int enteredId = scannerObj.nextInt();
        scannerObj.nextLine();
        System.out.print("Enter Goods Name: ");
        String enteredName = scannerObj.nextLine();
        System.out.print("Enter Goods Category: ");
        String enteredCategory = scannerObj.nextLine();
        goodsList[index] = new Goods(enteredId, enteredName, enteredCategory);
    }
    System.out.print("\nEnter Goods ID to Search: ");
    int searchKey = scannerObj.nextInt();
    Goods linearOutput = linearFind(goodsList, searchKey);
    System.out.println("\nLinear Search Result:");
    if (linearOutput != null) {
        linearOutput.displayGoods();
    } else {
        System.out.println("Goods not found.");
    }
}

```

```

        Goods[] arrangedGoods = Arrays.copyOf(goodsList, goodsList.length);
        Arrays.sort(arrangedGoods, Comparator.comparingInt(data -> data.goodsId));
        Goods binaryOutput = binaryFind(arrangedGoods, searchKey);
        System.out.println("\nBinary Search Result:");
        if (binaryOutput != null) {
            binaryOutput.displayGoods();
        } else {
            System.out.println("Goods not found.");
        }
        scannerObj.close();
    }
}

```

Output:

```

Linear Search Result:
Goods ID : 2
Goods Name : hp victus
Goods Category : laptops

Binary Search Result:
Goods ID : 2
Goods Name : hp victus
Goods Category : laptops

C:\Users\rehaa\OneDrive\Documents\sasank\cognizant>

```

Suitability:

Binary Search is more Suitable since there are more than 1000 documents containing the Products details so Binary Search works Faster. Seeing the Complexity difference the Linear Search works $O(n)$ whereas Binary Search works $O(\log n)$. So its better to use Binary Search only limitation that we got is that the Elements of the Search to be in Sorted order so that the Search happens Efficiently...

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. Understand Recursive Algorithms:

- Explain the concept of recursion and how it can simplify certain problems.

2. Setup:

- Create a method to calculate the future value using a recursive approach.

3. Implementation:

- Implement a recursive algorithm to predict future values based on past growth rates.

4. Analysis:

- Discuss the time complexity of your recursive algorithm.
- Explain how to optimize the recursive solution to avoid excessive computation.

Solution:

Recursion:

It is a programming technique in which a method calls itself to solve a problem. We break a large problem into smaller sub problems until all conditions are satisfied. Recursion can simplify problems that have repetitive or hierarchical structures.

```

import java.util.Scanner;
public class FinancePredictor {
    static double calculateFutureAmount(double presentAmount, double yearlyIncreaseRate, int totalYears) {
        if (totalYears == 0) {
            return presentAmount;
        }
        return calculateFutureAmount(
            presentAmount * (1 + yearlyIncreaseRate / 100),
            yearlyIncreaseRate,
            totalYears - 1
        );
    }
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("Financial Forecast Tool");

        System.out.print("Enter current amount: ");
        double startingAmount = inputScanner.nextDouble();

        System.out.print("Enter annual growth rate (%): ");
        double annualRate = inputScanner.nextDouble();

        System.out.print("Enter number of years: ");
        int yearsCount = inputScanner.nextInt();

        double predictedAmount = calculateFutureAmount(startingAmount, annualRate, yearsCount);

        System.out.printf("\nPredicted value after %d years: %.2f\n", yearsCount, predictedAmount);

        inputScanner.close();
    }
}

```

Output:

```

C:\Users\rehaa\OneDrive\Documents\sasank\cognizant>java FinancePredictor
Financial Forecast Tool
Enter current amount: 700000
Enter annual growth rate (%): 1.5
Enter number of years: 03

Predicted value after 3 years: 731974.86

```

Complexity:

For every year : $T(n) = T(n-1) + O(1)$

Time and Space Complexity will be $O(n)$. Where as n is number of years.

Optimization:

The recursive solution can be optimized by: Iterative approach or by Direct formula

This eliminates repeated recursive calls and improves efficiency for large values of years.